UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet-5

Name: TANUJ JOSHI
UID: 25MCA20081
Branch: MCA General
Section/Group: 25MCA_KAR-1
Semester: II
Date of Performance: 03-03-26
Subject Name: Technical Training
Subject Code: 25CAP-652

Aim of the Session:

The aim of this practical session is to implement and analyze cursors in SQL for row-by-row processing in a database. This helps in sequentially accessing and manipulating query results, enabling complex business logic such as payroll adjustments, dynamic calculations, and record-specific operations.

Objective of the Session:

The objectives of this practical session are:

- To fetch rows one by one from a result set using cursors.
- To perform conditional and procedural operations on individual records.
- To manage cursor lifecycle: DECLARE, OPEN, FETCH, CLOSE, and DEALLOCATE.
- To handle exceptions and prevent memory leaks during row-wise processing.
- To understand real-world applications like generating reports, updating salaries, and data migration.

Practical / Experiment Steps:

Step 1: Implementing a Simple Forward-Only Cursor

- Creating a cursor to loop through an Employee table and print individual records.

Step 2: Complex Row-by-Row Manipulation

- Using a cursor to update salaries based on a dynamic "Experience-to-Performance" ratio logic.

Step 3: Exception and Status Handling

- Ensuring the cursor handles empty result sets or termination signals gracefully.

Procedure of the Practical:

1. Start the system and log in to the computer.
2. Open the PostgreSQL client tool (psql / pgAdmin).
3. Create a table to store employee details for cursor demonstration.

```
CREATE TABLE Employee (
   EmpID SERIAL PRIMARY KEY,
   EmpName VARCHAR(50),
   Experience INT,
   Performance CHAR(1),
   Salary DECIMAL(10,2)
);
```

4. Insert sample records into the table.

```
INSERT INTO Employee (EmpName, Experience, Performance, Salary) VALUES
('Amit', 6, 'A', 50000),
('Riya', 4, 'B', 40000),
('Kunal', 7, 'A', 60000),
('Sneha', 3, 'B', 35000),
('Rahul', 2, 'C', 30000),
('Anita', 5, 'A', 55000);
```

Data Output Messages Notifications

	empid [PK] integer	empname character varying (50)	experience integer	performance character (1)	salary numeric (10,2)
1	1	Amit	6	A	50000.00
2	2	Riya	4	B	40000.00
3	3	Kunal	7	A	60000.00
4	4	Sneha	3	B	35000.00
5	5	Rahul	2	C	30000.00
6	6	Anita	5	A	55000.00

5. Implementing a Simple Forward-Only Cursor.

```
DO $$
DECLARE
   emp_rec RECORD;
   employee_cursor CURSOR FOR
      SELECT empid, empname, salary FROM employee;
BEGIN
   OPEN employee_cursor;
   LOOP
      FETCH employee_cursor INTO emp_rec;
      EXIT WHEN NOT FOUND;
      RAISE NOTICE 'ID: %, Name: %, Salary: %',
               emp_rec.empid, emp_rec.empname, emp_rec.salary;
   END LOOP;
   CLOSE employee_cursor;
END $$;
```

Data Output | Messages | Notifications

```
NOTICE:  ID: 1, Name: Amit, Salary: 50000.00
NOTICE:  ID: 2, Name: Riya, Salary: 40000.00
NOTICE:  ID: 3, Name: Kunal, Salary: 60000.00
NOTICE:  ID: 4, Name: Sneha, Salary: 35000.00
NOTICE:  ID: 5, Name: Rahul, Salary: 30000.00
NOTICE:  ID: 6, Name: Anita, Salary: 55000.00
DO
```

6. Complex Row-by-Row Manipulation.

```
DO $$
DECLARE
   emp_rec RECORD;
   employee_cursor CURSOR FOR
      SELECT empid, experience, performance, salary FROM employee;
BEGIN
   OPEN employee_cursor;
   LOOP
      FETCH employee_cursor INTO emp_rec;
      EXIT WHEN NOT FOUND;
      IF emp_rec.experience > 5 AND emp_rec.performance = 'A' THEN
         UPDATE employee SET salary = salary * 1.2 WHERE empid = emp_rec.empid;
      ELSIF emp_rec.experience > 3 THEN
         UPDATE employee SET salary = salary * 1.1 WHERE empid = emp_rec.empid;
```

```
        ELSE
            UPDATE employee SET salary = salary * 1.05 WHERE empid = emp_rec.empid;
        END IF;
    END LOOP;
    CLOSE employee_cursor;
END $$;
```

Data Output Messages Notifications

	empid [PK] integer	empname character varying (50)	experience integer	performance character (1)	salary numeric (10,2)
1	1	Amit	6	A	60000.00
2	2	Riya	4	B	44000.00
3	3	Kunal	7	A	72000.00
4	4	Sneha	3	B	36750.00
5	5	Rahul	2	C	31500.00
6	6	Anita	5	A	60500.00

7. Verify the output after execution.
8. Note down the results obtained.
9. Save the work and take screenshots for record.

I/O Analysis:

Input Provided:

- Cursor queries with DECLARE, OPEN, FETCH, CLOSE, DEALLOCATE.
- Employee sample data for row-wise operations.

Output Generated:

- Printed employee details row by row.
- Updated salaries based on conditions.
- Handled exceptions and ensured memory deallocation.

Learning Outcomes:

- Understanding sequential access of records using cursors.
- Mastering cursor lifecycle and syntax in SQL.

- Ability to perform conditional row-level operations.
- Awareness of exception handling and memory management in row-by-row processing.
- Practical exposure to real-world scenarios like payroll updates, reporting, and data migrations.